EX NO:13 **Node.js, Express.js and MongoDB**

DATE:

Problem Statement:

Develop a REST API using Node.js and Express.js to manage student records stored in MongoDB. The application should establish a database connection using Mongoose and implement CRUD operations for student records.Student Management REST API

App.jsx

```
import { useEffect, useState } from "react";

function App() {

  const [students, setStudents] = useState([]);

  const [name, setName] = useState("");

  const [department, setDepartment] = useState("");

  const [year, setYear] = useState("");

  const [editId, setEditId] = useState(null);

  // Get students

  const getStudents = () =>

    { fetch("http://localhost:3000/api/students

    ")

      .then(res => res.json())

      .then(data => setStudents(data));

  };

  useEffect(() =>

    { getStudents();

  }, []);

  // Add / Update student

  const saveStudent = () => {
```

```
const student =
  { name: name,
  department: department,
  year: Number(year)
};
if (editId)
  { fetch(`http://localhost:3000/api/students/${editId}`,
  {
    method: "PUT",
    headers: {
      "Content-Type": "application/json" },
    body: JSON.stringify(student)
  })
    .then(res => res.json())
    .then(() =>
      { getStudents();
      clearForm();
    });
} else {
  fetch("http://localhost:3000/api/students",
    { method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(student)
  })
```

```
            .then(res => res.json())
            .then(() =>
               { getStudents();
               clearForm();
            });
    }
};
// Delete student
const deleteStudent = (id) =>
    { fetch(`http://localhost:3000/api/students/${id}`,
    {
       method: "DELETE"
    })
       .then(res => res.json())
       .then(() =>
          { getStudents();
       });
};
// Edit student
const editStudent = (student) =>
    { setEditId(student._id);
    setName(student.name);
    setDepartment(student.department);
    setYear(student.year);
};
// Clear form
```

```
  const clearForm = () =>
    { setEditId(null);
    setName("");
    setDepartment("");
    setYear("");
  };
  return (
    <div>
      <h1>Student Management</h1>
      <input
        type="text"
        placeholder="Student Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />
      <input
        type="text"
        placeholder="Department"
        value={department}
        onChange={(e) => setDepartment(e.target.value)}
      />
      <input
        type="number"
        placeholder="Year"
        value={year}
```

```
            onChange={(e) => setYear(e.target.value)}
          />
          <button onClick={saveStudent}>
            {editId ? "Update Student" : "Add Student"}
          </button>
          <button onClick={clearForm}>
            Clear
          </button>
          <hr />
          <h2>Student List</h2>
          {students.map(student => (
            <div key={student._id}>
              <p>
                {student.name} - {student.department} - Year {student.year}
                <button onClick={() => editStudent(student)}>
                  Edit
                </button>
                <button onClick={() => deleteStudent(student._id)}>
                  Delete
                </button>
              </p>
            </div>
          ))}
        </div>
    );
```

```
}
export default App;
```

Student.js

```
const mongoose = require("mongoose");
const studentSchema = new
    mongoose.Schema({ name: String,
    department: String,
    year: Number
});
module.exports = mongoose.model("Student", studentSchema);
```

StudentRoutes.js

```
const express = require("expres");
const router = express.Router();
const {
    getStudents,
    getStudent,
    createStudent,
    updateStudent,
    deleteStudent
} = require("../controllers/studentController");
router.get("/", getStudents);
router.get("/:id", getStudent);
router.post("/", createStudent);
router.put("/:id", updateStudent);
router.delete("/:id", deleteStudent);
```

```
module.exports = router;
```

StudentController.js

```
const Student = require("../models/Student");
// Get all students
exports.getStudents = async (req, res) =>
  { try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
// Get student by ID
exports.getStudent = async (req, res) =>
  { try {
    const student = await Student.findById(req.params.id);
    if (!student) {
      return
        res.status(404).json({ message:
        "Student not found"
      });
    }
    res.json(student);
  } catch (error)
    { res.status(400).json({
      message: "Invalid student ID"
```

```
        });
    }
};
// Add student
exports.createStudent = async (req, res) =>
    { try {
        const student = new
            Student({ name: req.body.name,
            department: req.body.department,
            year: req.body.year
        });
        const savedStudent = await student.save();
        res.status(201).json(savedStudent);
    } catch (error)
        { res.status(500).json({
            message: error.message
        });
    }
};
// Update student
exports.updateStudent = async (req, res) =>
    { try {
        const student = await
            Student.findByIdAndUpdate( req.params.id,
            {
```

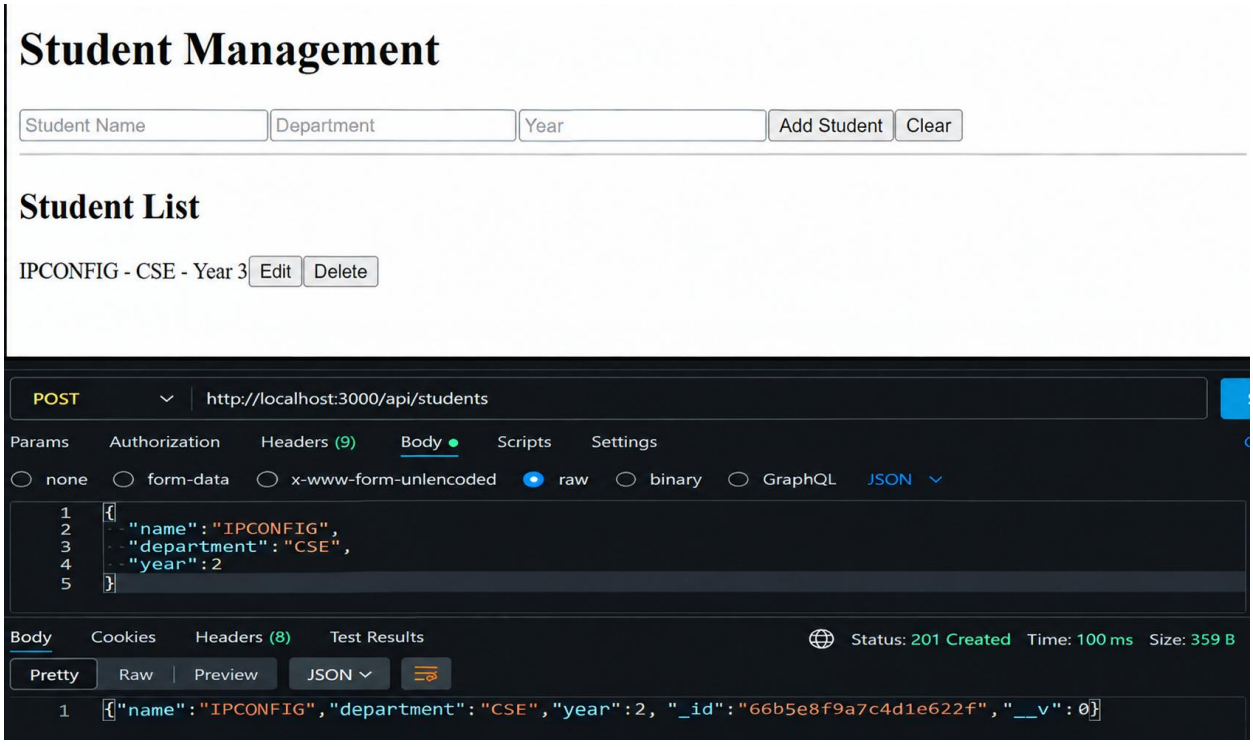
```
            name: req.body.name,
            department: req.body.department,
            year: req.body.year
        },
        { new: true }
      );
      if (!student) {
        return
          res.status(404).json({ message:
          "Student not found"
        });
      }
      res.json(student);
    } catch (error)
      { res.status(400).json({
        message: "Invalid student ID"
      });
    }
};
// Delete student
exports.deleteStudent = async (req, res) =>
    { try {
      const student = await Student.findByIdAndDelete(req.params.id);
      if (!student) {
        return
          res.status(404).json({ message:
          "Student not found"
```

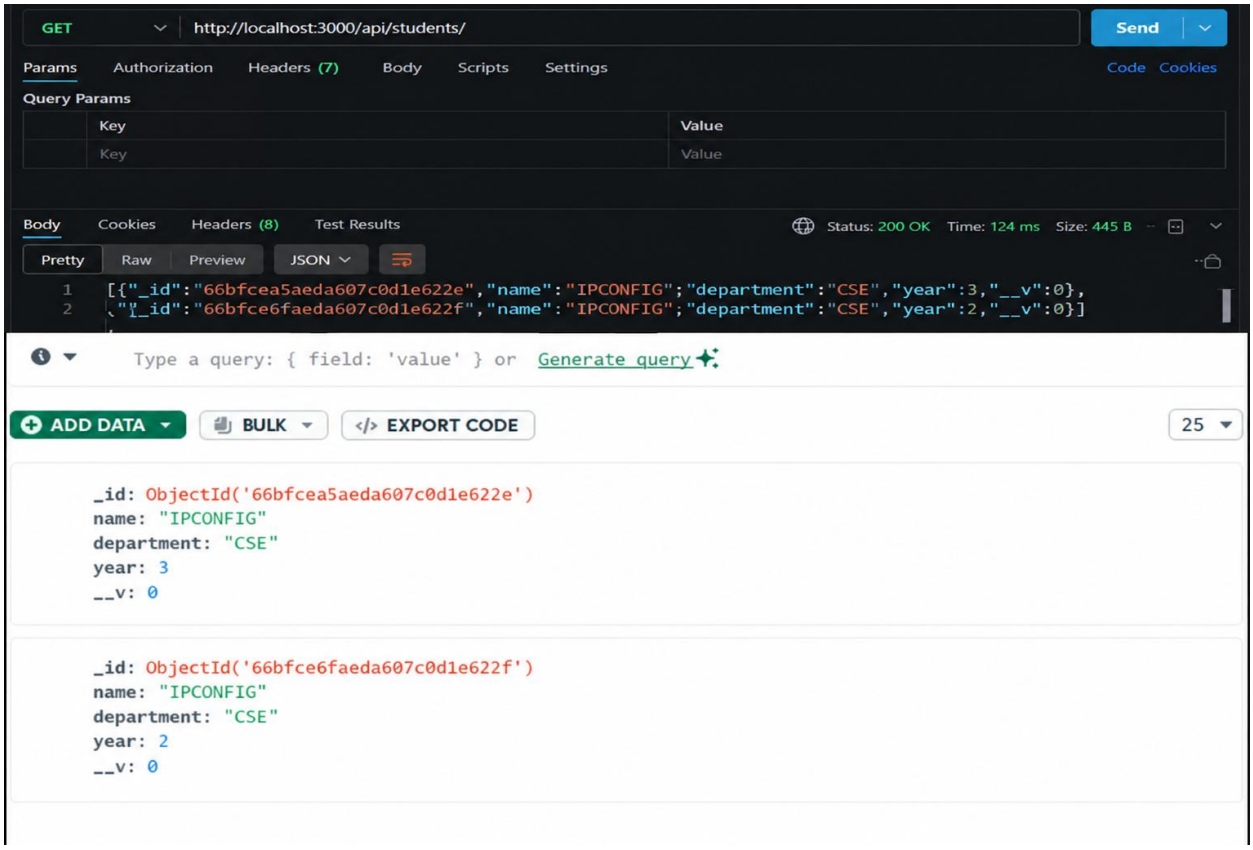
```
        });
    }
    res.json({
        message: "Student deleted successfully"
    });
  } catch (error)
    { res.status(400).json({
        message: "Invalid student ID"
    });
  }
};
```

MONGO_URI=mongodb+srv://sarathikamal2006_db_user:Sarathi2026@cluster0.kws3wur.mongodb.net/studentdb

PORT=3000

Output:





Problem Understand & Approach (10)	Implementation and Execution (15)	Tool/ Technology usage (10)	Code Quality & Documentation (5)	Viva (5)	Time Management (5)	Total (50)

RESULT:

Thus, the Student Management application was successfully developed using React, Node.js, Express.js, and MongoDB with MVC architecture. The application implemented CRUD operations for student records, with the Model managing MongoDB data, the Controller handling application logic, and the Routes managing API requests. The React frontend was connected to the Express backend to add, display, and delete student records.